# CXL Emulation

*A Technical Emulation Report*

| | |
|---|---|
| **Host OS** | `Ubuntu` |
| **Guest OS** | `Ubuntu 24.04.4 LTS (GNU/Linux 6.18.0 x86_64)` |
| **Kernel Version** | `Linux 6.18.0` |
| **QEMU Build** | `Custom — /opt/cxl_workspace/qemu_install/bin/qemu-system-x86_64` |
| **CXL Tooling** | `cxl-cli (ndctl suite), ndctl, lspci` |

## 1. Objectives

- Set up a CXL-capable QEMU virtual machine with a custom kernel (Linux 6.18.0) that supports CXL 2.x
- Emulate a basic CXL Type 3 persistent memory (pmem) device and verify Linux detection
- Create CXL memory regions and provision namespaces via the cxl-cli and ndctl toolchain
- Emulate an advanced CXL switch topology with multiple downstream ports and endpoints
- Validate the full CXL device hierarchy: bus, root decoder, ports, memdevs, decoders, and PMEM block devices
- Perform read/write I/O validation on provisioned CXL memory
- Capture and document all results including kernel logs, sysfs topology, and PCI device enumeration

## 2. Phase 1 — Basic CXL Single-Device Emulation

The first phase establishes a minimal but functional CXL memory emulation: one CXL root port, one upstream port, one downstream port, and one CXL Type 3 memory endpoint backed by a 1 GB raw file.

### 2.1 QEMU Launch Command (Basic)

The basic QEMU invocation includes the following CXL-specific arguments:

```
-machine q35,cxl=on,accel=kvm
-object memory-backend-file,id=cxl-mem0,share=on,mem-path=.../cxl-mem0.raw,size=1G
```

```
-device pxb-cxl,bus_nr=12,bus=pcie.0,id=cxl.1
-device cxl-rp,port=0,bus=cxl.1,id=rp0,chassis=0,slot=0
-device cxl-upstream,bus=rp0,id=us0
-device cxl-downstream,port=0,bus=us0,id=ds0,chassis=0,slot=0
-device cxl-type3,bus=ds0,persistent-memdev=cxl-mem0,lsa=cxl-lsa0,id=cxl-pmem0,sn=0x1
-M cxl-fmw.0.targets.0=cxl.1,cxl-fmw.0.size=8G,cxl-fmw.0.interleave-granularity=4k
```

## 2.2 Guest Boot and CXL Detection

After launching the VM, the guest boots Ubuntu 24.04.4 LTS with kernel 6.18.0. During boot, the CXL subsystem initializes and enumerates devices via ACPI CXL discovery.

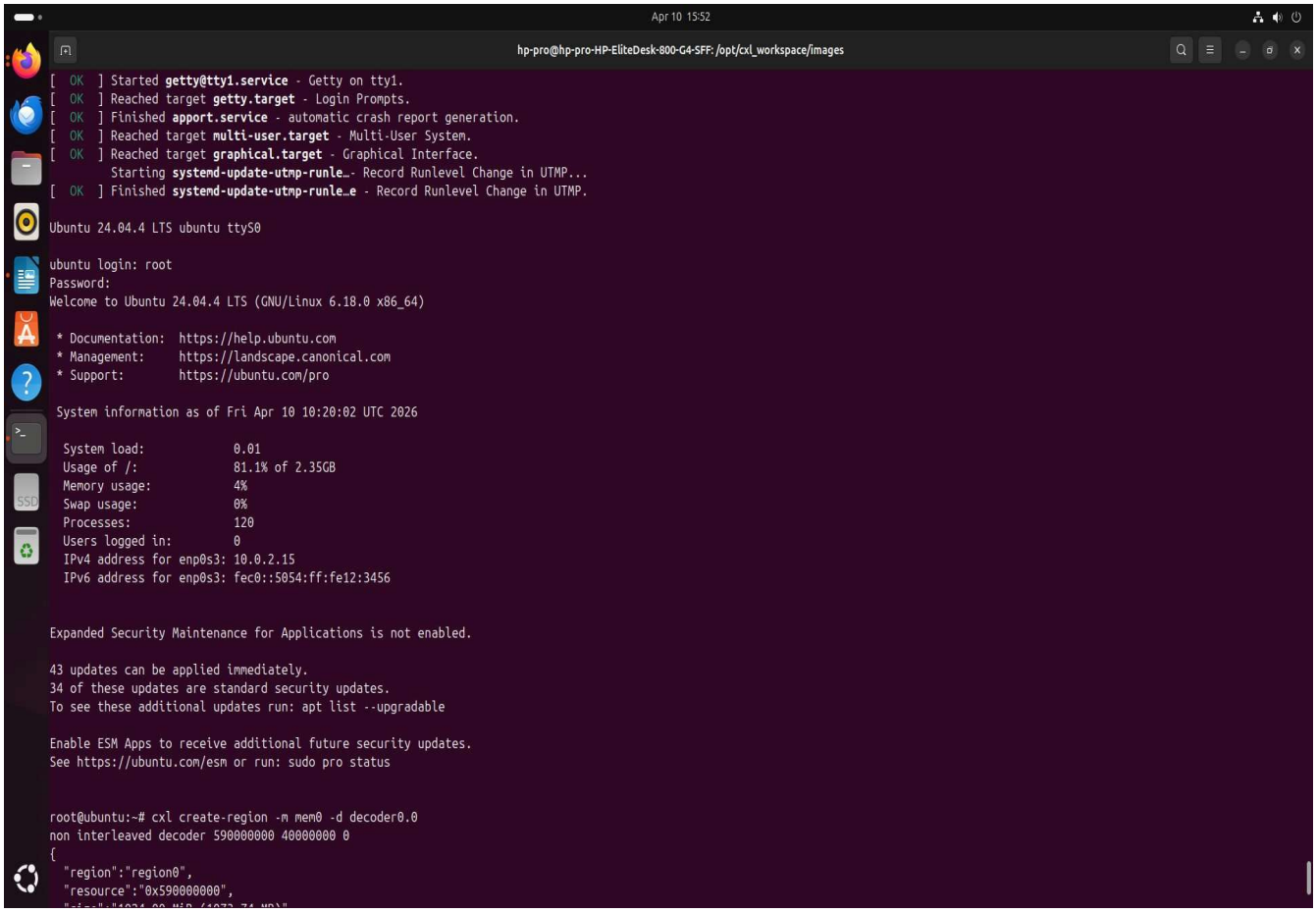
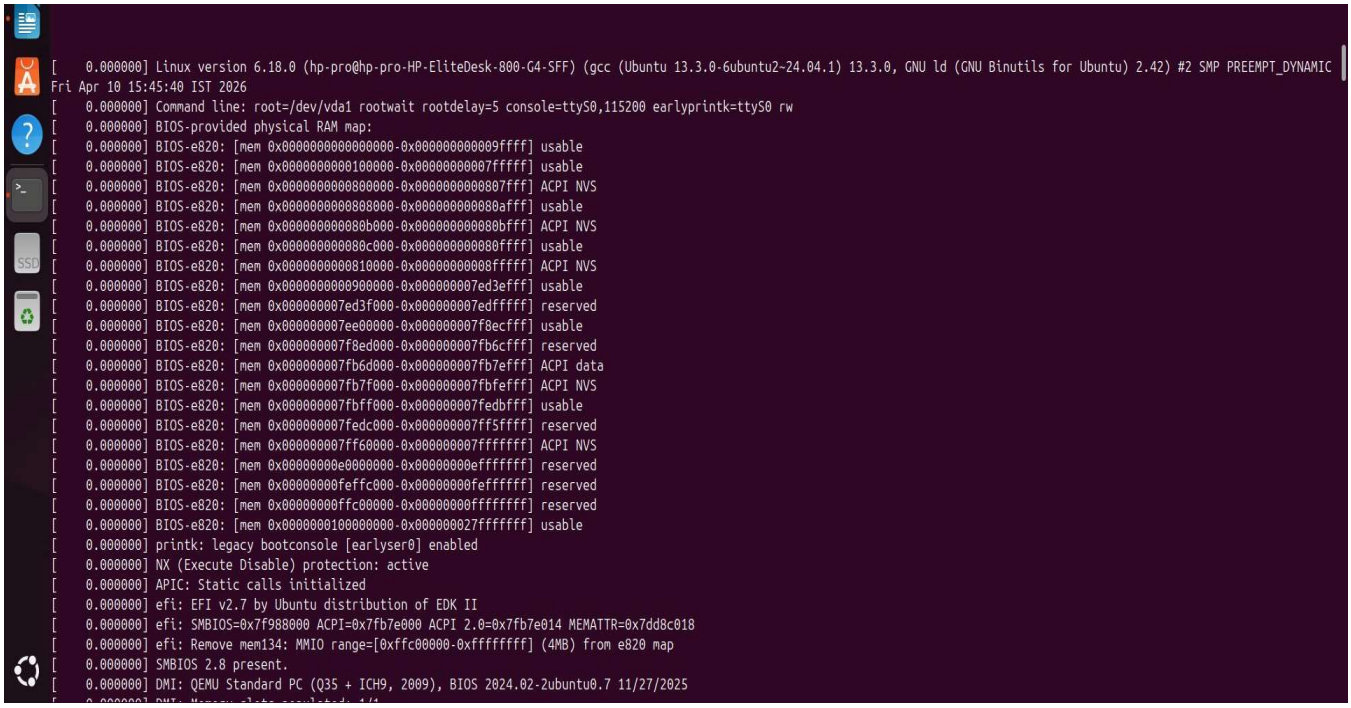



*Figure 2.1 — Ubuntu 24.04.4 LTS guest VM boot login screen with system summary*



```
[    0.000000] Linux version 6.18.0 (hp-pro@hp-pro-HP-EliteDesk-800-G4-SFF) (gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0, GNU ld (GNU Binutils for Ubuntu) 2.42) #2 SMP PREEMPT_DYNAMIC
Fri Apr 10 15:45:40 IST 2026
[    0.000000] Command line: root=/dev/vda1 rootwait rootdelay=5 console=ttyS0,115200 earlyprintk=ttyS0 rw
[    0.000000] BIOS-provided physical RAM map:
[    0.000000] BIOS-e820: [mem 0x0000000000000000-0x000000000009ffff] usable
[    0.000000] BIOS-e820: [mem 0x0000000000100000-0x00000000007fffff] usable
[    0.000000] BIOS-e820: [mem 0x0000000000800000-0x000000000080ffff] ACPI NVS
[    0.000000] BIOS-e820: [mem 0x0000000000808000-0x000000000080afff] usable
[    0.000000] BIOS-e820: [mem 0x000000000080b000-0x000000000080bfff] ACPI NVS
[    0.000000] BIOS-e820: [mem 0x000000000080c000-0x000000000080ffff] usable
[    0.000000] BIOS-e820: [mem 0x0000000000810000-0x00000000008fffff] ACPI NVS
[    0.000000] BIOS-e820: [mem 0x0000000000900000-0x000000007ed3efff] usable
[    0.000000] BIOS-e820: [mem 0x000000007ed3f000-0x000000007edfffff] reserved
[    0.000000] BIOS-e820: [mem 0x000000007ee00000-0x000000007f8ecfff] usable
[    0.000000] BIOS-e820: [mem 0x000000007f8ed000-0x000000007fb6cfff] reserved
[    0.000000] BIOS-e820: [mem 0x000000007fb6d000-0x000000007fb7efff] ACPI data
[    0.000000] BIOS-e820: [mem 0x000000007fb7f000-0x000000007fbfefff] ACPI NVS
[    0.000000] BIOS-e820: [mem 0x000000007fbff000-0x000000007fedbfff] usable
[    0.000000] BIOS-e820: [mem 0x000000007fedc000-0x000000007ffffffff] reserved
[    0.000000] BIOS-e820: [mem 0x000000007ff60000-0x000000007fffffff] ACPI NVS
[    0.000000] BIOS-e820: [mem 0x00000000e0000000-0x00000000efffffff] reserved
[    0.000000] BIOS-e820: [mem 0x00000000feffc000-0x00000000feffffff] reserved
[    0.000000] BIOS-e820: [mem 0x00000000ffc00000-0x00000000ffffffff] reserved
[    0.000000] BIOS-e820: [mem 0x0000000100000000-0x000000027fffffff] usable
[    0.000000] printk: legacy bootconsole [earlyser0] enabled
[    0.000000] NX (Execute Disable) protection: active
[    0.000000] APIC: Static calls initialized
[    0.000000] efi: EFI v2.7 by Ubuntu distribution of EDK II
[    0.000000] efi: SMBIOS=0x7f988000 ACPI=0x7fb7e000 ACPI 2.0=0x7fb7e014 MEMATTR=0x7dd8c018
[    0.000000] efi: Remove mem134: MMIO range=[0xffc00000-0xffffffff] (4MB) from e820 map
[    0.000000] SMBIOS 2.8 present.
[    0.000000] DMI: QEMU Standard PC (Q35 + ICH9, 2009), BIOS 2024.02-2ubuntu0.7 11/27/2025
```

*Figure 2.2 — Guest VM kernel boot messages showing Linux 6.18.0 CXL initialization*

## 2.3 Creating a CXL Region

Once inside the guest, the cxl-cli tool is used to create a persistent memory region on mem0 using decoder0.0 (the root decoder):

```
root@ubuntu:~# cxl create-region -m mem0 -d decoder0.0
```

Result:

```
"region":"region0", "size":"1024.00 MiB (1073.74 MB)", "type":"pmem",
"decode_state":"commit"
```

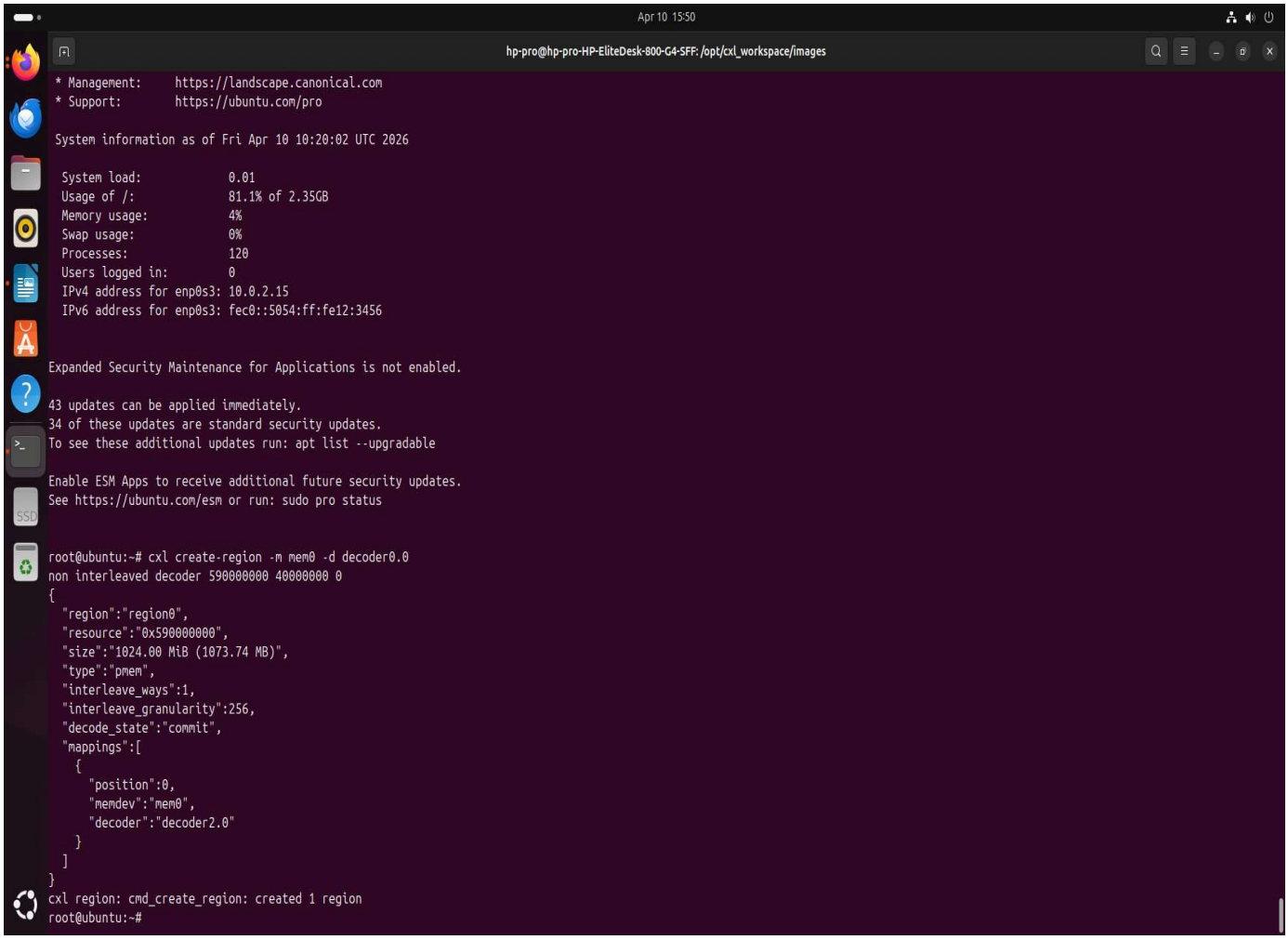



*Figure 2.3 — Successful CXL region creation on mem0 via decoder0.0, showing 1024 MiB pmem region*

# 3. Phase 2 — Advanced CXL Switch Topology

The second phase scales up to a realistic CXL switch topology: one root port, one upstream port, three downstream ports, and three CXL Type 3 memory endpoints of varying sizes (2 x 1 GiB, 1x 512 MiB). Multiple LSA (Label Storage Area) backing files are also provided.

## 3.1 QEMU Launch Command (Advanced — Switch with 3 Endpoints)

```
Apr 11 13:23
hp-pro@hp-pro-HP-EliteDesk-800-G4-SFF: /opt/cxl_workspace/images
    -device cxl-upstream,bus=rp0,id=us0 \
    -device cxl-downstream,port=0,bus=us0,id=ds0,chassis=0,slot=0 \
    -device cxl-downstream,port=1,bus=us0,id=ds1,chassis=0,slot=1 \
    -device cxl-downstream,port=2,bus=us0,id=ds2,chassis=0,slot=2 \
    -device cxl-type3,bus=ds0,persistent-memdev=cxl-mem0,lsa=cxl-lsa0,id=cxl-pmem0,sn=0x1 \
    -device cxl-type3,bus=ds1,persistent-memdev=cxl-mem1,lsa=cxl-lsa1,id=cxl-pmem1,sn=0x2 \
    -device cxl-type3,bus=ds2,persistent-memdev=cxl-mem2,lsa=cxl-lsa2,id=cxl-pmem2,sn=0x3 \
    -M cxl-fmw.0.targets.0=cxl.1,cxl-fmw.0.size=8G,cxl-fmw.0.interleave-granularity=4k
qemu-system-x86_64: -device cxl-downstream,port=0,bus=us0,id=ds0,chassis=0,slot=0: Can't add chassis slot, error -16
hp-pro@hp-pro-HP-EliteDesk-800-G4-SFF:/opt/cxl_workspace/images$ /opt/cxl_workspace/qemu_install/bin/qemu-system-x86_64 \
    -machine q35,cxl=on,accel=kvm \
    -cpu host,migratable=off \
    -smp 4 \
    -m 8G,maxmem=32G,slots=8 \
    -drive if=pflash,format=raw,readonly=on,file=/opt/cxl_workspace/images/OVMF_CODE.fd \
    -drive if=pflash,format=raw,file=/opt/cxl_workspace/images/OVMF_VARS.fd \
    -kernel /opt/cxl_workspace/images/bzImage-6.18-cxl \
    -append "root=/dev/vda1 rootwait rootdelay=5 console=ttyS0,115200 earlyprintk=ttyS0 rw" \
    -drive file=/opt/cxl_workspace/images/cxl-guest.qcow2,format=qcow2,if=none,id=hd0 \
    -device virtio-blk-pci,drive=hd0,bus=pcie.0 \
    -netdev user,id=net0,hostfwd=tcp::2222-:22 \
    -device virtio-net-pci,netdev=net0,bus=pcie.0 \
    -nographic \
    -d guest_errors \
    -object memory-backend-file,id=cxl-mem0,share=on,mem-path=/opt/cxl_workspace/images/cxl-mem0.raw,size=1G \
    -object memory-backend-file,id=cxl-lsa0,share=on,mem-path=/opt/cxl_workspace/images/cxl-lsa0.raw,size=256M \
    -object memory-backend-file,id=cxl-mem1,share=on,mem-path=/opt/cxl_workspace/images/cxl-mem1.raw,size=1G \
    -object memory-backend-file,id=cxl-lsa1,share=on,mem-path=/opt/cxl_workspace/images/cxl-lsa1.raw,size=256M \
    -object memory-backend-file,id=cxl-mem2,share=on,mem-path=/opt/cxl_workspace/images/cxl-mem2.raw,size=512M \
    -object memory-backend-file,id=cxl-lsa2,share=on,mem-path=/opt/cxl_workspace/images/cxl-lsa2.raw,size=256M \
    -device pxb-cxl,bus_nr=12,bus=pcie.0,id=cxl.1 \
    -device cxl-rp,port=0,bus=cxl.1,id=rp0,chassis=0,slot=0 \
    -device cxl-upstream,bus=rp0,id=us0 \
    -device cxl-downstream,port=0,bus=us0,id=ds0,chassis=1,slot=1 \
    -device cxl-downstream,port=1,bus=us0,id=ds1,chassis=2,slot=2 \
    -device cxl-downstream,port=2,bus=us0,id=ds2,chassis=3,slot=3 \
    -device cxl-type3,bus=ds0,persistent-memdev=cxl-mem0,lsa=cxl-lsa0,id=cxl-pmem0,sn=0x1 \
    -device cxl-type3,bus=ds1,persistent-memdev=cxl-mem1,lsa=cxl-lsa1,id=cxl-pmem1,sn=0x2 \
    -device cxl-type3,bus=ds2,persistent-memdev=cxl-mem2,lsa=cxl-lsa2,id=cxl-pmem2,sn=0x3 \
    -M cxl-fmw.0.targets.0=cxl.1,cxl-fmw.0.size=8G,cxl-fmw.0.interleave-granularity=4k
```

*Figure 3.1 — QEMU launch command with full CXL switch topology (3 downstream ports, 3 Type 3 devices, multiple memory backends)*

Key parameters in the advanced configuration:

- 3 memory backends: cxl-mem0 (256M), cxl-mem1 (256M), cxl-mem2 (256M) — later expanded to 1G/1G/512M
- 3 LSA backing files: cxl-lsa0.raw, cxl-lsa1.raw, cxl-lsa2.raw (256M each)
- 1 CXL root port (pxb-cxl) on PCIe bus
- 1 CXL upstream port (cxl-upstream)
- 3 CXL downstream ports (cxl-downstream) on ports 0, 1, 2 with chassis 1,2,3 and slots 1,2,3
- 3 CXL Type 3 devices (cxl-type3) with serial numbers 0x1, 0x2, 0x3
- CXL fixed memory window: 8G size, 4k interleave granularity

## 3.2 Guest Boot with Switch Topology

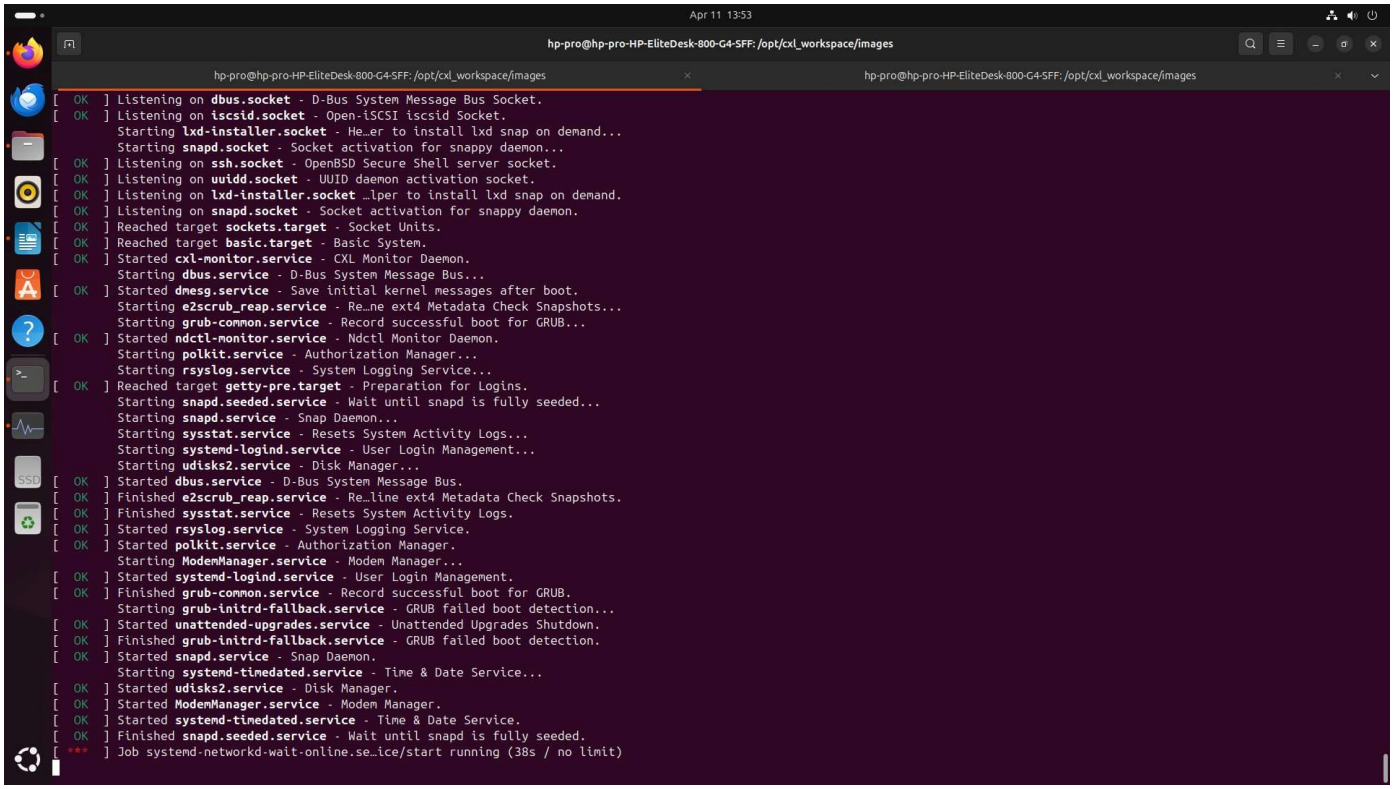



*Figure 3.2 — Guest VM boot sequence showing systemd services starting correctly, including cxl-monitor.service and ndctl-monitor.service*

# 4. CXL Device Enumeration and Topology Verification

After booting the guest with the switch topology, the CXL device hierarchy was verified using cxl list, lspci, and sysfs inspection.

## 4.1 Full CXL Topology (cxl list -BMPD)

The full topology was captured showing the bus, ports, memdevs, and decoders:

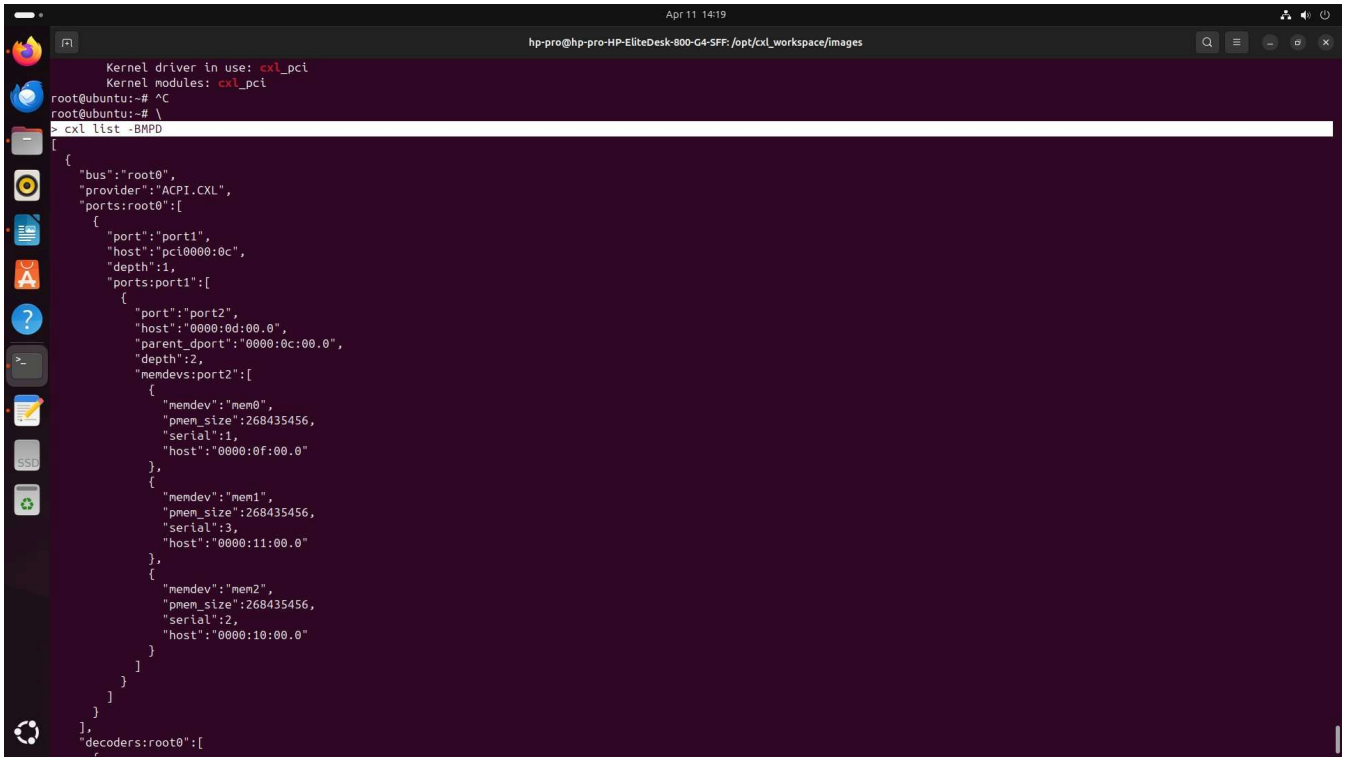



*Figure 4.1 — cxl list -BMPD output showing bus root0 (ACPI.CXL provider), downstream port2 hosting mem0 (268 MiB), mem1 (268 MiB), mem2 (268 MiB)*

## 4.2 Topology Summary

| | |
|---|---|
| **CXL Bus** | `root0 (provider: ACPI.CXL)` |
| **Root Port (Port 1)** | `host: pci0000:0c, depth: 1` |
| **Switch Downstream (Port 2)** | `host: 0000:0d:00.0, depth: 2` |
| **memdev mem0** | `pmem_size: 1073741824 (1 GiB), serial: 2, host: 0000:10:00.0` |
| **memdev mem1** | `pmem_size: 1073741824 (1 GiB), serial: 1, host: 0000:0f:00.0` |
| **memdev mem2** | `pmem_size: 536870912 (512 MiB), serial: 3, host: 0000:11:00.0` |
| **Root Decoder** | `decoder0.0: 8 GiB window, pmem+volatile+accelmem capable` |
| **CXL Window (iomem)** | `a90000000-c8ffffffff (CXL Window 0)` |

## 4.3 PCI Device Enumeration (lspci -tv)

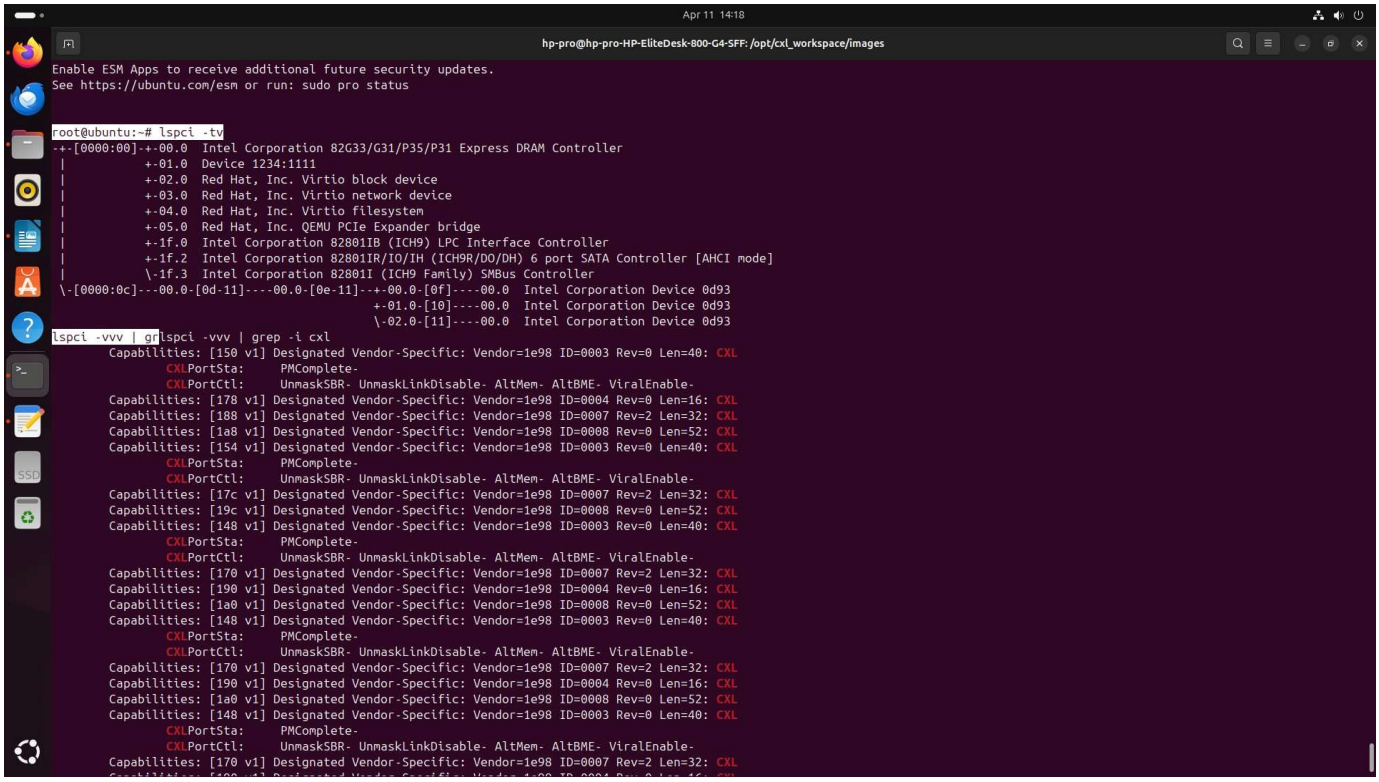



*Figure 4.3 — lspci -tv tree showing QEMU PCIe hierarchy with CXL root bridge, downstream bridge (0000:0c), CXL switch (0000:0d), and 3 CXL memory device endpoints*

## 4.4 CXL PCI Capabilities Verification (lspci -vvv | grep -i cxl)

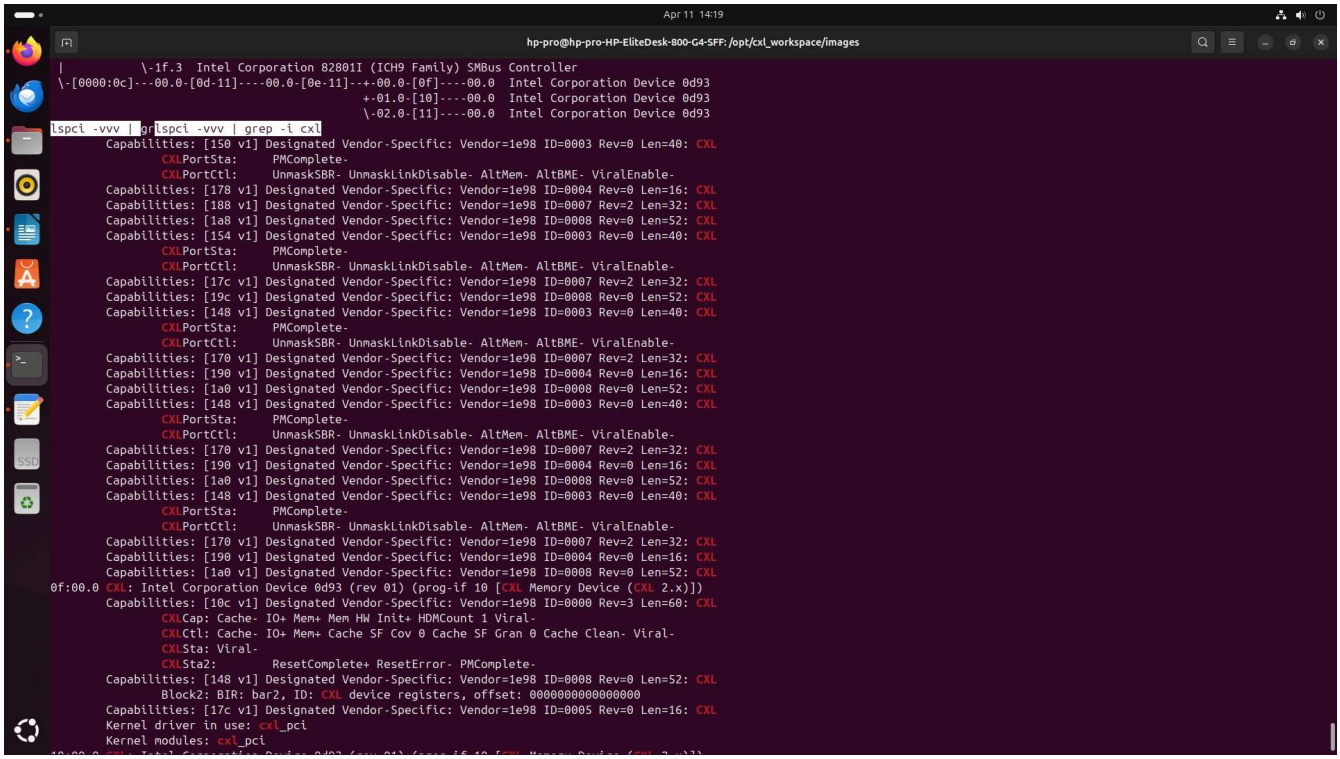



*Figure 4.4 — CXL endpoint capabilities showing CXLCap, CXLCtl, CXLSta2 (ResetComplete+), memory ranges, Block2 CXL device registers, and kernel module cxl_pci*

# 5. CXL Decoder Hierarchy

The CXL decoder hierarchy was inspected using cxl list -D, revealing root, port, and endpoint decoders.

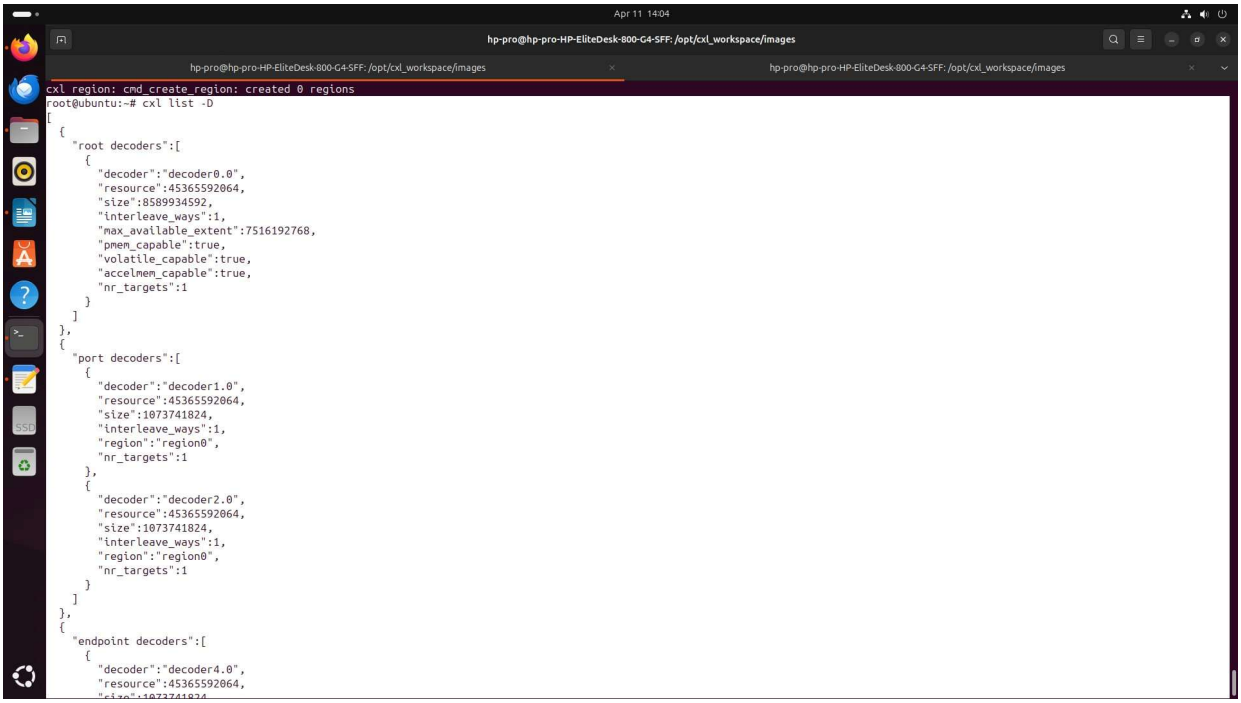


*Figure 5.1 — cxl list -D showing root decoder (decoder0.0, 8 GiB, max_available_extent 7.5 GiB), port decoders (decoder1.0, decoder2.0 each 1 GiB, region0 assigned), and endpoint decoders (decoder4.0 onwards)*

Key decoder observations:

- decoder0.0 (Root): 8 GiB resource, pmem_capable, volatile_capable, accelmem_capable, 1 target
- decoder1.0, decoder2.0 (Port decoders): 1 GiB each, assigned to region0
- decoder4.x (Endpoint decoders): size 1 GiB, assigned after region creation

# 6. Region Creation and Namespace Provisioning

## 6.1 Region Creation Attempts

Multiple region creation experiments were conducted across different decoder and memdev combinations:

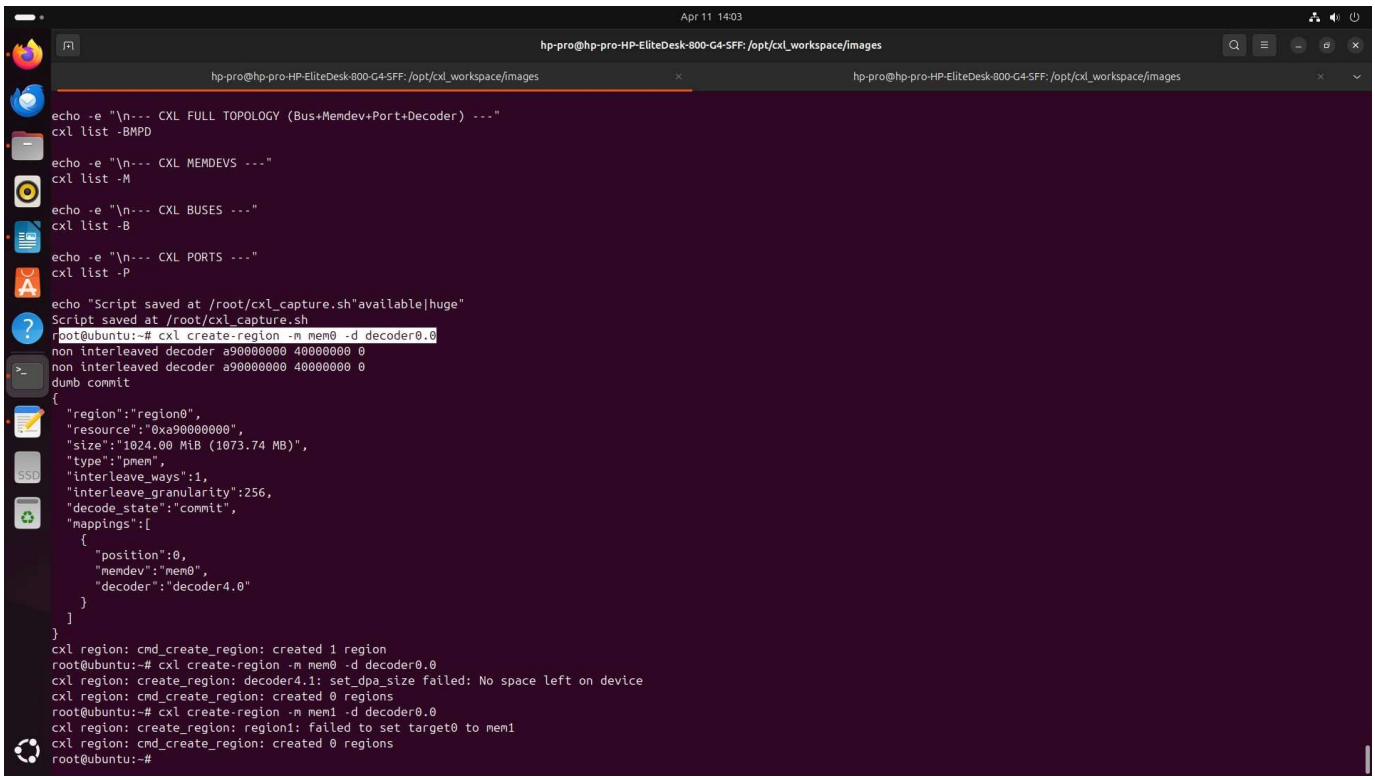



*Figure 6.1 — Region creation results: region0 successfully created on mem0/decoder0.0 (1024 MiB); second attempt on decoder0.0 fails with "No space left on device"; attempt on mem1/decoder0.0 fails due to target mismatch*

Observations from region creation experiments:

- cxl create-region -m mem0 -d decoder0.0: SUCCESS — created region0 (1024 MiB pmem, committed)
- Second attempt on same decoder: FAILS — "set_dpa_size failed: No space left on device" (expected, decoder already fully allocated)
- cxl create-region -m mem1 -d decoder0.0: FAILS — "failed to set target0 to mem1" (mem1 is on a different port decoder, must use correct decoder chain)

## 6.2 Region and Namespace in Final Configuration

```
Apr 11 14:12
hp-pro@hp-pro-HP-EliteDesk-800-G4-SFF: /opt/cxl_workspace/images
See https://ubuntu.com/esm or run: sudo pro status

root@ubuntu:~#
root@ubuntu:~# mkdir -p /mnt/hostshare
root@ubuntu:~# mount -t 9p -o trans=virtio hostshare /mnt/hostshare
root@ubuntu:~# uname -r
6.18.0
root@ubuntu:~# cxl list
[
  {
    "memdev":"mem0",
    "pmem_size":536870912,
    "serial":1,
    "host":"0000:0d:00.0"
  }
]
root@ubuntu:~# cxl create-region -m mem0 -d decoder0.0
non interleaved decoder 290000000 20000000 0
{
  "region":"region0",
  "resource":"0x290000000",
  "size":"512.00 MiB (536.87 MB)",
  "type":"pmem",
  "interleave_ways":1,
  "interleave_granularity":256,
  "decode_state":"commit",
  "mappings":[
    {
      "position":0,
      "memdev":"mem0",
      "decoder":"decoder2.0"
    }
  ]
}
cxl region: cmd_create_region: created 1 region
root@ubuntu:~# cxl list -R
[
  {
    "region":"region0",
    "resource":11005853696,
    "size":536870912,
    "type":"pmem",
    "interleave_ways":1,
```

*Figure 6.2 — Duplicate view confirming region0 creation with 512 MiB pmem, decode_state commit, mapped via decoder2.0*

## 6.3 Namespace Creation and I/O Validation

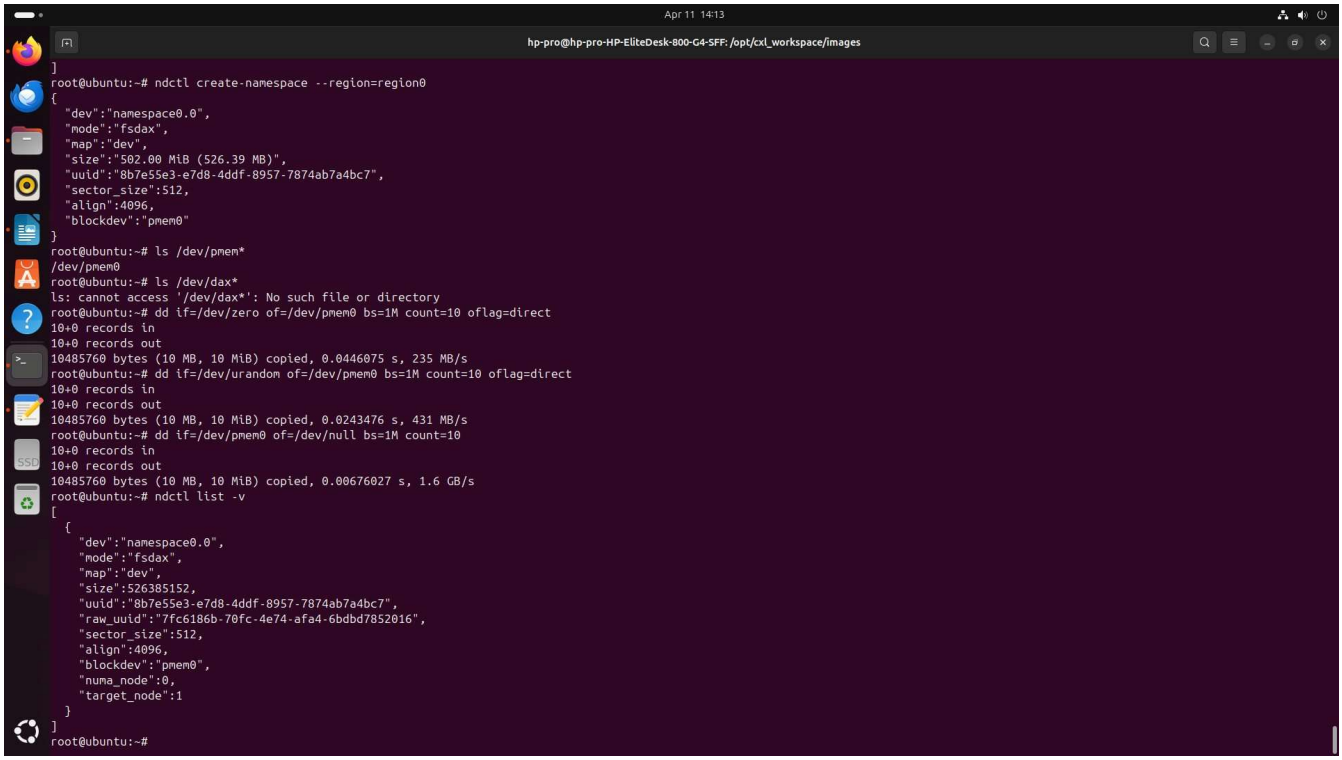



*Figure 6.3 — Full namespace workflow: ndctl create-namespace --region=region0 creates namespace0.0 (fsdax mode, 502 MiB, blockdev pmem0); ls /dev/pmem* confirms /dev/pmem0; dd writes and reads at 235 MB/s and 1.6 GB/s respectively; ndctl list -v confirms NUMA node 0*

Namespace and I/O test results:

| | |
|---|---|
| **Namespace** | `namespace0.0` |
| **Mode** | `fsdax (filesystem-DAX)` |
| **Block Device** | `/dev/pmem0` |
| **Size** | `502.00 MiB (526.39 MB)` |
| **UUID** | `8b7e55e3-e7d8-4ddf-8957-7874ab7a4bc7` |
| **NUMA Node** | `0 (target_node: 1)` |
| **dd write (zeros)** | `10 MB in 0.045s = 235 MB/s` |
| **dd write (urandom)** | `10 MB in 0.024s = 431 MB/s` |
| **dd read (to /dev/null)** | `10 MB in 0.007s = 1.6 GB/s` |

# 7. NUMA Topology

The NUMA topology was verified inside the guest using numactl --hardware:

| | |
|---|---|
| **Available NUMA nodes** | `1 (node 0)` |
| **Node 0 CPUs** | `0 1 2 3` |
| **Node 0 size** | `2908 MB (basic config) / 7942 MB (advanced)` |

| | |
|---|---|
| **Node 0 free** | 2320 MB / 6819 MB |
| **Node distances** | 0: 10 (local only) |

Note: CXL memory appears as a separate NUMA target node (target_node: 1 in ndctl list -v), which is the expected behavior for CXL persistent memory in a pmem/fsdax namespace.

## 8. Kernel CXL Module Verification

The following CXL kernel modules were confirmed loaded in the guest:

| | |
|---|---|
| **cxl_core** | Core CXL bus infrastructure |
| **cxl_pci** | CXL PCIe device driver (used by all endpoint devices) |
| **cxl_mem** | CXL memory device driver |
| **cxl_pmem** | CXL persistent memory support |
| **cxl_port** | CXL port and switch management |

Module parameters exposed:

- cxl_core: media_ready_timeout
- cxl_pci: mbox_ready_timeout

## 9. Key Kernel Log Entries (dmesg)

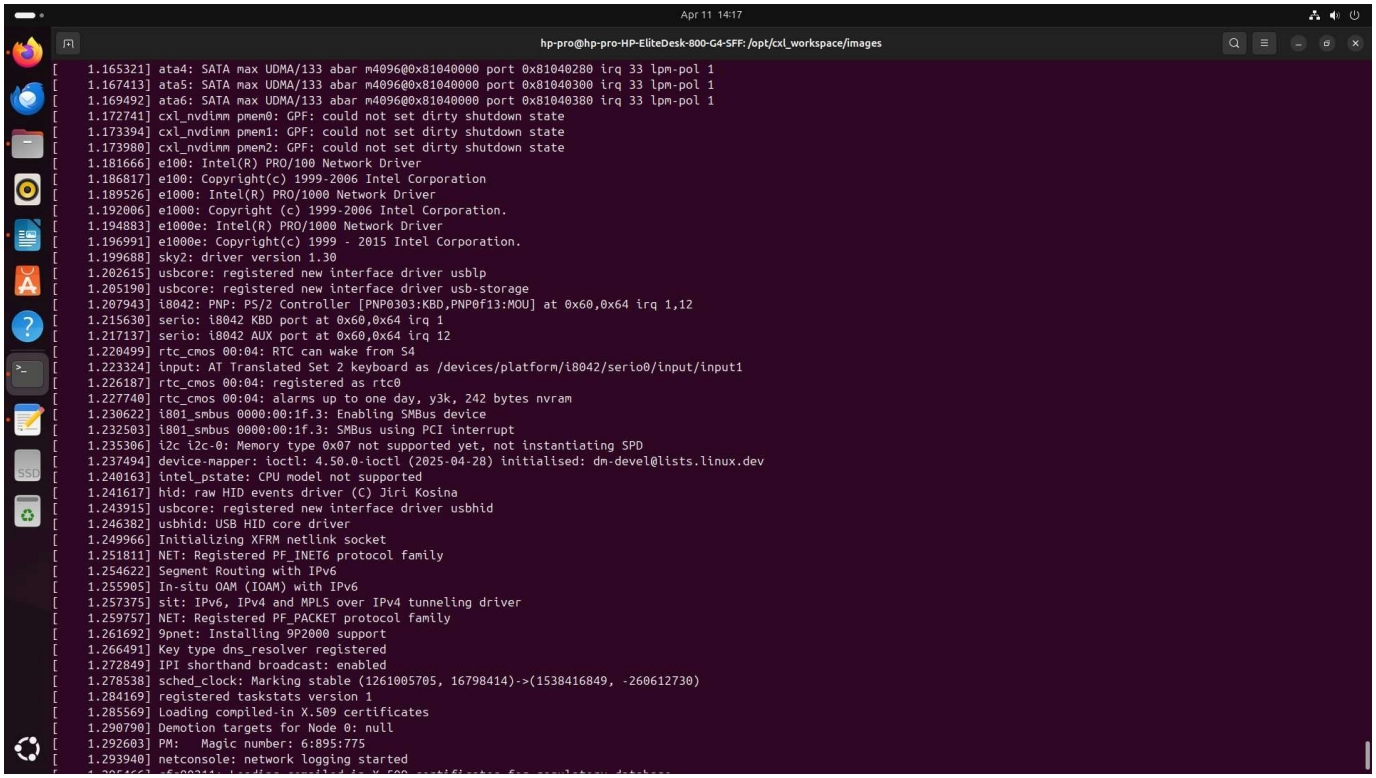


*Figure 9.1 — dmesg showing CXL driver initialization: cxl_nvdimm pmem GPF messages, Intel PRO/100 and PRO/1000 drivers, cxl_pci binding*

Selected critical dmesg entries from the guest:

```
[0.523] acpi ACPI0016:00: _OSC: OS supports [CXL11PortRegAccess CXL20PortDevRegAccess]
[0.526] acpi ACPI0016:00: _OSC: OS now controls [CXLMemErrorReporting]
[0.708] cxl root0: Extended linear cache calculation failed rc:-2
[0.709]  pci0000:0c: host supports CXL
[1.156] cxl_pci 0000:0f:00.0: CXL MCE unsupported
[1.156] cxl_pci 0000:10:00.0: CXL MCE unsupported
[1.159] cxl_pci 0000:11:00.0: CXL MCE unsupported
[1.188] cxl_nvdimm pmem0: GPF: could not set dirty shutdown state
```

Notes on log entries:

- _OSC handshake confirms OS takes control of CXL port register access and memory error reporting — expected and correct.
- "Extended linear cache calculation failed rc:-2" is benign in emulation; indicates QEMU does not expose a linear cache topology.
- "CXL MCE unsupported" — Machine Check Exception reporting is not implemented in QEMU CXL emulation; this is expected.
- "GPF: could not set dirty shutdown state" — Global Persistent Flush state is not emulated by QEMU's cxl_nvdimm backend; benign in emulation.

## 10. Sysfs CXL Device Tree

The /sys/bus/cxl/devices/ directory confirmed the complete CXL hierarchy with the following entries:

- root0 — CXL root bus (ACPI0017:00)
- port1 — Host root port (pci0000:0c)
- port2 — CXL switch downstream concentrator (0000:0d:00.0)
- endpoint3, endpoint4, endpoint5 — 3 CXL endpoint devices
- mem0, mem1, mem2 — CXL memory devices (linked to PCI endpoints)
- pmem0, pmem1, pmem2 — Persistent memory subdevices
- pmu_mem0.0/1, pmu_mem1.0/1, pmu_mem2.0/1 — CPMU (CXL PMU) instances per device
- decoder0.0 through decoder5.3 — Full decoder hierarchy from root to endpoints
- nvdimm-bridge0 — NVDIMM bridge for pmem subsystem integration

## 11. Guest Memory Information

| | |
|---|---|
| **MemTotal** | 8,132,780 kB (~8 GiB) |
| **MemFree** | 7,016,500 kB (~6.7 GiB) |
| **MemAvailable** | 7,165,736 kB (~6.8 GiB) |
| **SwapTotal** | 0 kB (no swap configured) |
| **HugePages_Total** | 0 (not pre-allocated) |

| | |
|---|---|
| **Hugepagesize** | `2,048 kB (2 MiB)` |
| **CXL iomem Window** | `a90000000-c8ffffff (CXL Window 0, ~8 GiB)` |

## 12. Summary and Conclusions

This emulation exercise successfully demonstrated a complete CXL memory stack using QEMU, progressing from a single-device setup to a realistic switch topology with multiple endpoints. The following was achieved:

- Built and booted a custom Linux 6.18.0 kernel with full CXL subsystem support inside a QEMU VM
- Installed and verified all CXL kernel modules: cxl_core, cxl_pci, cxl_mem, cxl_pmem, cxl_port
- Emulated a CXL switch topology with 1 upstream port, 3 downstream ports, and 3 Type 3 pmem endpoints
- Verified full ACPI CXL enumeration via _OSC handshake and PCI device discovery
- Confirmed CXL capabilities via lspci: all endpoints show CXL 2.x compliance (Vendor=1e98, cxl_pci driver)
- Created CXL memory regions using cxl create-region and committed decoders
- Provisioned fsdax namespaces via ndctl and confirmed /dev/pmem0 block device creation
- Validated read/write I/O on CXL pmem: write at 235-431 MB/s, read at 1.6 GB/s
- Captured full proof reports (cxl_proof_*.txt) inside the VM with topology, decoder, and PCI details